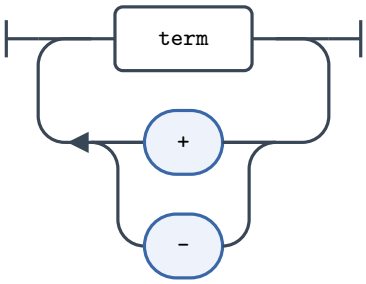
Syntax Diagram Generator — EBNF examples

Each example below pairs an EBNF grammar (ISO/IEC 14977) with the railroad diagram the generator produces for every one of its rules. The diagrams are the tool's own TikZ output.

1 Concatenation with a repeated separator

```
expression = term, { ("+" | "-"), term };
```

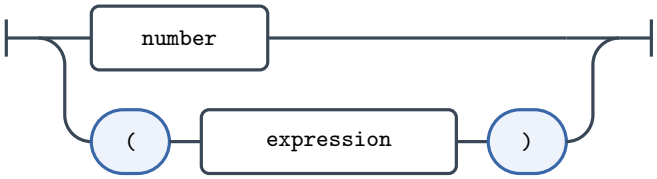
expression



2 Alternation and grouping

```
factor = number | "(", expression, ")";
```

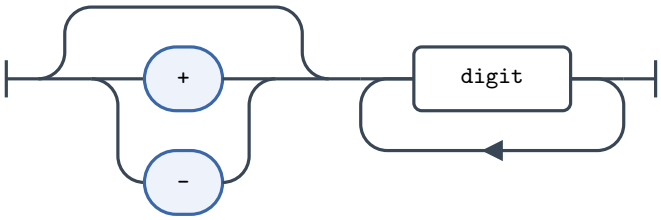
factor



3 Optional and one-or-more

```
signed number = [ "+" | "-" ], digit, { digit };
```

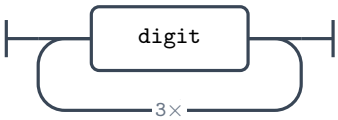
signed number



4 Repetition factor

```
triplet = 3 * digit;
```

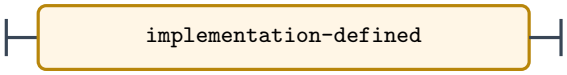
triplet



5 Special sequence

```
token = ? implementation-defined ? ;
```

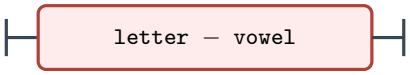
token



6 Exception

```
consonant = letter - vowel ;
```

consonant

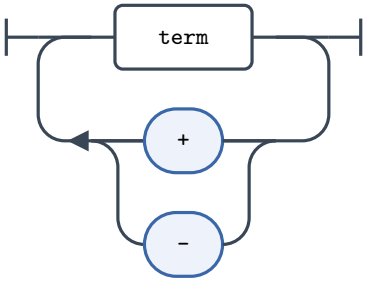


7 A complete grammar (multiple rules)

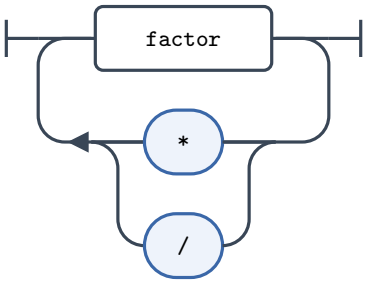
```
(* A small arithmetic grammar (ISO/IEC 14977 EBNF) *)

expression = term, { ("+" | "-"), term };

term = factor, { ("*" | "/"), factor };
```

```
factor = number
       | "(", expression, ")";

number = digit, { digit };

digit = "0" | "1" | "2" | "3" | "4"
      | "5" | "6" | "7" | "8" | "9";
```

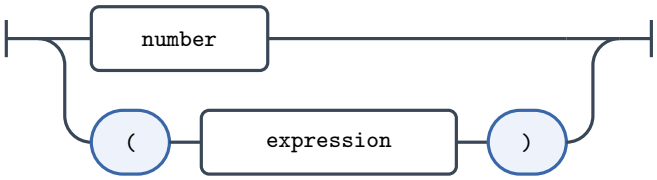
expression



term

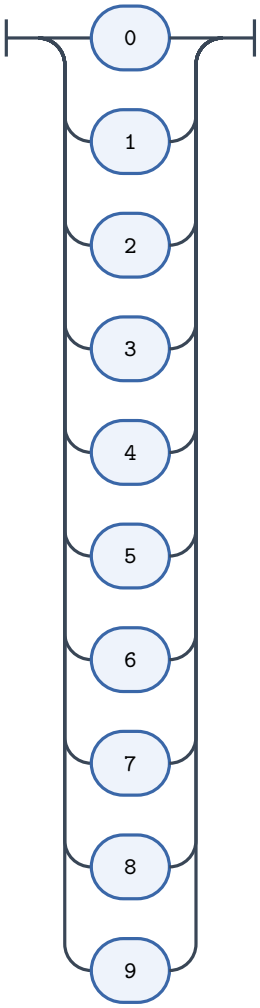


factor



number

digit

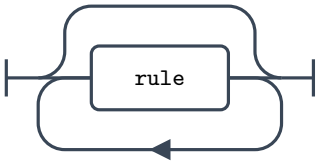


8 EBNF described in EBNF

```
(* EBNF described in EBNF (after ISO/IEC 14977) *)

grammar = { rule };

rule = identifier, "=", definitions list, ";";

definitions list = single definition, { "|", single definition };

single definition = term, { ",", term };
```

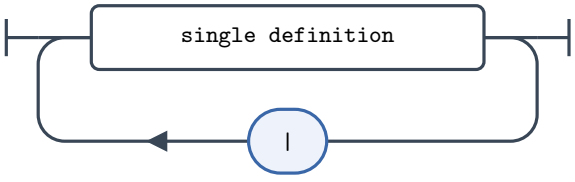
```
term = factor, [ "-", factor ];

factor = [ integer, "*" ], primary;

primary = optional sequence
        | repeated sequence
        | grouped sequence
        | identifier
        | terminal string
        | special sequence;

optional sequence = "[", definitions list, "]";

repeated sequence = "{", definitions list, "}";

grouped sequence = "(", definitions list, ")";

terminal string = '"', character, { character }, '"'
                | "'", character, { character }, "'";

special sequence = "?", { character }, "?";

identifier = letter, { letter | digit | " " };

integer = digit, { digit };
```

grammar

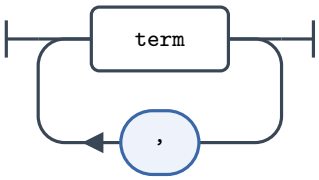


rule

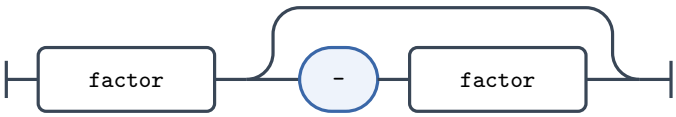
definitions list



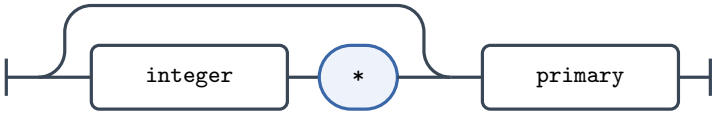
single definition



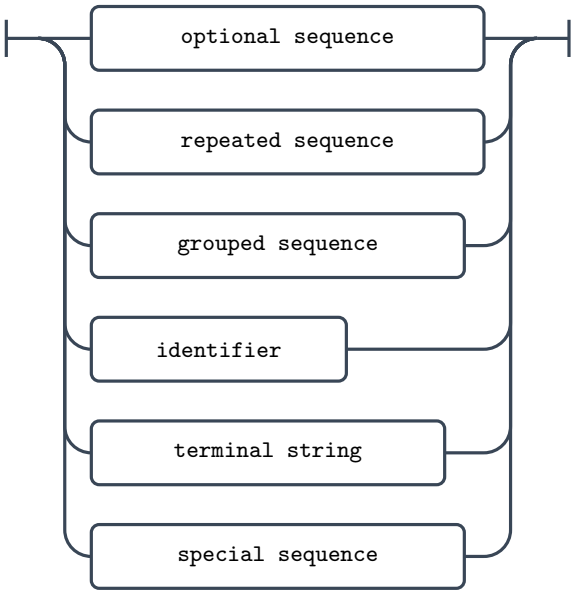
term



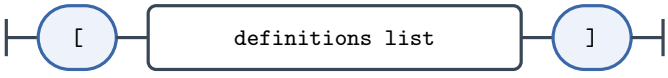
factor



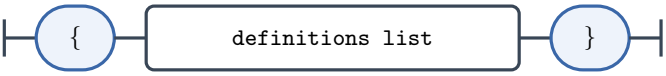
primary



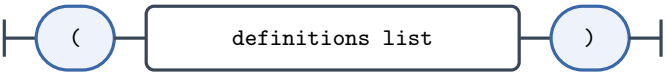
optional sequence



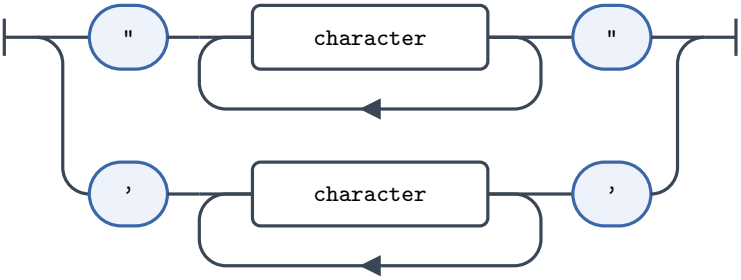
repeated sequence



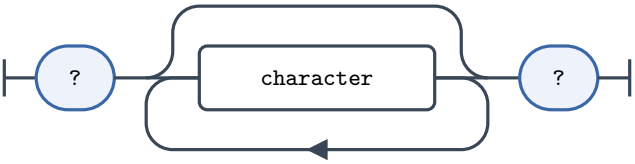
grouped sequence



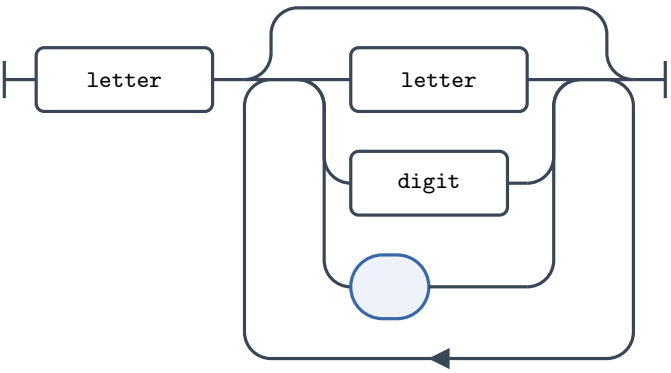
terminal string



special sequence



identifier



integer


digit